Local Search Methods for Discrete Optimization

Optimization is everywhere! Every day, you decide how to allocate your time and money. Cities decide where to build schools, libraries, and hospitals. Airlines decide how to schedule their planes and their crews. Doctors decide upon treatment plans. Etc.

These decisions are typically made with some objective in mind, or perhaps competing objectives that involve tradeoffs. For example, how do you balance your time and money?

If you are not on Brown's meal plan, you might regularly decide between cooking dinner for yourself or eating out. Cooking dinner takes longer, but it is less expensive. Maybe your solution is invest in a dishwasher, so that you can cook for yourself, but save time cleaning up. This is a solution to an optimization problem.

How does AI solve optimization problems? The next two units focus on algorithms for both discrete and continuous optimization. Problems are discrete when the possible choices are discrete (e.g., integers, paths, matchings), such as which workers should be assigned to which shifts, or which students should be assigned to which schools. Problems are continuous when the decisions are real-valued quantities, such as how much electricity to generate, or what parameter settings are best for our machine-learning algorithm?

1 Discrete Optimization

A **discrete** optimization problem is one in which the state space is discrete (e.g., the integers, paths through a graph, etc.). The following features are typical of the discrete optimization problems we will study:

- **combinatorial** state space (such as paths through a graph, schedules, assignments, etc.)
- cost function to be minimized or reward function to be maximized
- similar solutions have similar objective values ("continuity")
- exhaustive, blind, and heuristic search are often intractable, so that optimal solutions are likely out of reach, but reasonable solutions are satisfactory—mostly, because they have to be!

Here are a few concrete examples of discrete optimization problems:

- **Integrated Circuit (IC) Design** Modern computer chips pack tens of billions of transistors onto a piece of silicon the size of a fingernail! How do they do it? And what does a good design embody?

  IC design involves some continuous variables, such as transistor size, supply voltage, and wire thickness. It also involves discrete variables, namely the placement of each transistor, and the paths among the wires that connect them. These choices are made so as to maximize a PPA objective: "P" for power consumption (less is better), "P" for performance (faster is better), and "A" for area (smaller is better). However, these desiderata are in conflict: minimizing area creates congestion which slows things down; but maximizing speed requires higher voltage, which increases power consumption.

  For years, IC design optimization problems were solved via local search, the topic of this lecture. But recently, reinforcement learning has shown great promise as an alternative solution method.

- **Facility Location** Suppose a city is deciding where to build its fire stations, knowing that fire trucks will be dispatched from these stations in emergencies. Intuitively, these stations should be spread out all around the city, so that at least one station covers every neighborhood. To formalize this problem, we must quantify how good a set of locations actually is. For example, we might seek to minimize average response time, i.e., how far from the nearest fire station each building in the city is, on average. Or perhaps we care about minimizing the *maximum* response time, so that no one is ever too far away from assistance. Given such an objective function, we can evaluate the quality of different arrangements of fire stations in the city in search of an optimal, or at least a preferred, solution.
- **Vehicle Routing** Every day, delivery companies, like FedEx and UPS, are contracted to deliver collections of packages to myriad locations. The vehicle routing problem is the problem of assigning packages to trucks (whose storage capacities are limited), and arranging routes for the truck drivers, so as to minimize the amount of time it takes to make all the deliveries (or the total fuel consumption, or the distance traveled, etc.). One curious fact is that UPS delivery routes avoid making left turns, whenever possible, as it was discovered that this heuristic leads to substantial time savings.

2 Formal Problem Definition

An **optimization problem** (X, f) comprises a state space X and an objective function $f : X \to \mathbb{R}$, in which the goal is to identify a state x^* that is either a (global) minimum or a maximum: i.e., it is an optimum. Given an optimization problem, $x^* \in X$ is a minimum iff $f(x^*) \leq f(x)$, for all $x \in X$; similarly, $x^* \in X$ is a maximum iff $f(x^*) \geq f(x)$, for all $x \in X$. This lecture is tailored toward minimization problems and algorithms; maximization problems are handled analogously.

As the search spaces in optimization problems can be exceedingly large, one approach to solving them is to search only locally: i.e., from a state to a neighboring state. To employ such an approach, an AI practitioner defines a **neighborhood function** $\mathcal{N} : X \rightrightarrows X$, which typically returns simple perturbations of x for which $f(x)$ can be computed efficiently. A state $x_l^* \in X$ is a local minimum iff $f(x_l^*) \leq f(y)$, for all $y \in \mathcal{N}(x_l^*)$. (And likewise, for a local maximum.)

The objective function of an optimization problem gives rise to a cost (or reward) landscape. As shown in Figure 1, in addition to its global optima, this landscape can include local optima, from which it may be difficult to escape, especially if many counterproductive steps are needed (i.e., uphill steps in the diagram), before productive steps (i.e., downhill steps) are identified. Flat regions present a further difficulty. Some flat regions may be global minima, where exploration beyond a local neighborhood may not be beneficial; but other flat regions, like the region labelled "shoulder" in Figure 1, may be worth escaping from.

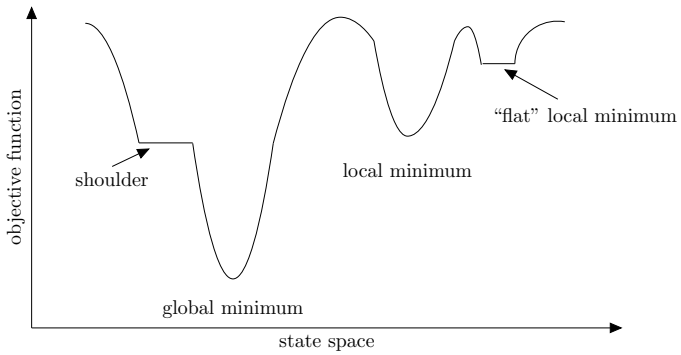


Figure 1: A one-dimensional state-space landscape. This figure is adopted from Russell & Norvig.

A search algorithm moves around such a landscape, from state to state (i.e., from one candidate solution to another), seeking local optima—the lowest troughs or highest peaks. **Local search** methods are algorithms that iteratively seek improvements. They repeatedly 1. perturb the current solution, and 2. update it if any of these perturbations yields an improvement. The term "local" refers to the fact that algorithms in this class only consider perturbations within a local neighborhood of the current solution.

Among local search methods, we study (i) hill-climbing algorithms, which greedily improve upon the current solution and (ii) simulated annealing, which temporarily explores changes worse than the current solution, but ultimately only exploits improvements.

A local search must begin at some initial state. One common approach to finding an initial state is—drum roll—heuristic search! In general, heuristic search can be slow. But it is also possible to run a quick-and-dirty heuristic search, such as a greedy search (without backtracking). Alternatively, local search can be initialized at random. In fact, local search can be restarted multiple times—or run multiple times in parallel—from random initial states, increasing the likelihood of eventually discovering a global optimum.

3 Search vs. Optimization

In contrast to typical search problems (e.g., Rubik's cube),[1] the solution to a *pure* optimization problem, discrete or continuous, does not include the path to a goal, but is only the goal itself. IC design, facility location, and vehicle routing are all pure optimization problems in this sense. Such problems naturally lend themselves to local search: i.e., starting from a complete solution, and improving that solution so long as resources remain available.

But even problems whose solutions involve paths, such as the traveling salesperson problem (see Section 4), can be solved via local search. By so doing, we forego the optimality guarantees of blind and heuristic search methods, but those methods may fail to produce any solution at all to very large-scale optimization problems. Local search methods are fast, and often produce solutions of decent quality, even if their optimality cannot be guaranteed.

	Search	Optimization
Start state	Empty solution	Complete solution
Actions	Extend the current solution	Improve the current solution
Weakness	Combinatorial explosion	Local optima
Strength	Optimality guarantees	Fast
Feasibility	By construction	Require checks E.g., alter a schedule by a swap; check all deadlines are still met
Methods	A*, Branch-n-bound	Hill-climbing, Simulated Annealing, Genetic Algorithms

4 Traveling Salesperson Problem

Figure 2 presents an instance of the traveling salesperson problem (TSP), a classic optimization problem. Providence, Boston, Hartford, New York, and Washington D.C., and their respective distances from one another (as the crow flies), are depicted. A path through the graph that visits all cities *exactly once* and returns to its origin is called a **tour**. The objective in TSP is to find a tour of minimal distance. The optimal tour in Figure 2 is PBHWNP, for which the total distance is 795.

[1] which might be more aptly named **planning** problems, since their solutions involve sequences of actions, *a.k.a.* **plans**

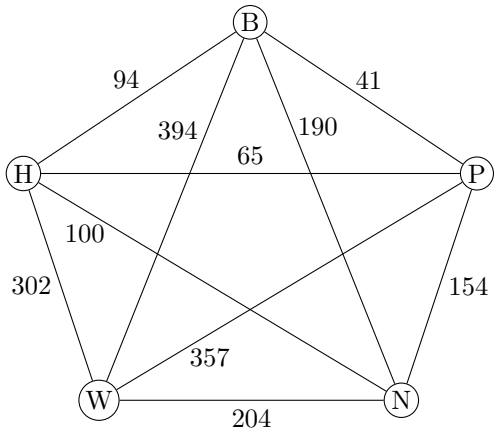


Figure 2: This graph encodes an instance of TSP in which states are tours, and the objective function f is defined as a tour's total distance, as indicated by the weights along its edges.

Given your experience with search so far, it would be quite natural for you to propose that we tackle TSP by formulating it as a search problem: i.e., by building up tours incrementally. In Appendix A, we describe two ways in which to formulate TSP as search. The difficulty with this approach is the number of tours: starting in Providence, there are 4 possible next cities, and from there, 3 possibilities, and so on. In general, on a complete graph of n nodes, there are $(n-1)!$ paths from any one node to all the others and back again.[2]

Instead of formulating TSP as search, we will take the point of view of optimization, where states are tours, and our job is to define a neighborhood operation based on which we can run local search.

Before proceeding, however, we remind you that local search methods must start somewhere! They require an initial state. Heuristic search (e.g., A* with backtracking) is too onerous for this purpose. Instead, we suggest using greedy search (without backtracking) to generate a quick-and-dirty tour that can be used as an initial state before embarking on a local search. For example, a greedy search might proceed by applying the **nearest-neighbor heuristic**: visit the nearest city next, as long as that city has not already been visited, until all cities have been visited, and then return home.

There are many possible neighborhood operations. One simple example is 2OPT: given a tour, pick two cities on the tour (other than the origin), and reverse the order in which all the cities between them, inclusive, are visited. 2OPT restricted to single hops on a tour is called **adjacent swap**. This heuristic simply swaps the order of any two consecutive cities on the tour. More formally, given a tour segment x_1, y_1, x_2, y_2, adjacent swap removes (x_1, y_1) and (x_2, y_2), and reconnects the nodes as (x_1, x_2) and (y_1, y_2). Adjacent swap, and 2OPT more generally, are useful heuristics in practice because their computation is cheap—just subtract the distances of the deleted edges and add the distances of the inserted ones.

How might local search behave on our sample TSP, with adjacent swap as the neighborhood operation? Suppose the initial state is PWBHNP of distance 1099. Applying adjacent swap yields PBWHNP of distance 991, PWHBNP of distance 1097, and PWBNHP, of distance 1106. Most likely, a local search algorithm would accept PBWHNP, since $991 < 1099$; but, it could also reasonably accept PWHBNP since $1097 < 1099$. Assume PBWHNP is accepted. Applying adjacent swap again yields PWBHNP (the original tour), PBHWNP of distance 795, and PBWNHP of distance 804. Now, if the tour PBHWNP is accepted, the algorithm verifies that it encountered a local (in fact, global) minimum and halts. (See Figure 3.)

[2]without repeats, i.e., without visiting the same city twice

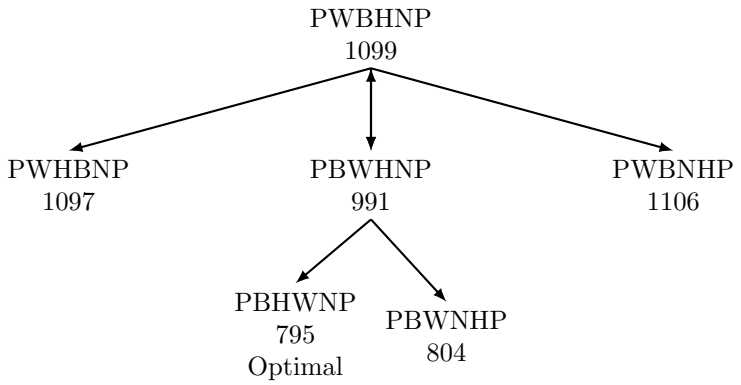


Figure 3: Local Search in TSP.

5 Hill-Climbing

Hill-climbing is a family of local search algorithms that consider changes in the local neighborhood of a given state and accept changes that improve upon the current solution. Best-improvement and first-improvement are examples of hill-climbing algorithms. Other (randomized) variants include GSAT and WalkSAT, which are special purpose algorithms for solving satisfiability.[3]

5.1 Best-Improvement Search

The main idea behind **best-improvement** search is to consider *all* states in the local neighborhood of the current state, and to accept one that best improves the objective function. Like all local search algorithms, best-improvement search converges to a local (but not necessarily global) minimum. And again, like all local search algorithms, its performance can be improved using random restarts.

5.2 First-Improvement Search

Unlike best-improvement search, **first-improvement** search does not consider all neighbors of the current solution. On the contrary, it considers its neighbors at random and accepts the first one that demonstrates an improvement. It halts if ever it loses its *patience* before finding an improvement. First-improvement search is also called **stochastic local search**.

Assuming infinite patience, stochastic local search converges to a local optimum with probability 1. But rather than run the algorithm with too much patience, it is usually run repeatedly with random restarts. Assuming infinite patience and an infinite number of random restarts s.t. every basin of attraction[4] is eventually an initial one, stochastic local search converges to a *global* optimum with probability 1.

[3] Coming soon, to a lecture hall near you!

[4] A **basin of attraction** is a set of starting states in a dynamical system that eventually converges to the same final state.

BestImprovementSearch	
Inputs	optimization problem $(X, f, \mathcal{N})$
	random start state x
Output	best state visited x
Initialize	$x' \neq x$, $A = \{x\}$

`while` $(x' \neq x)$ `do`

1. $x' = x$, $A = \{x\}$
2. for all $y \in \mathcal{N}(x')$
   (a) if $f(y) < f(x)$, then $x = y$ and $A = \{x\}$
   (b) else if $f(y) = f(x)$, then insert y in A
3. choose x in A

`return` x

Table 1: Best-Improvement Search.

6 Simulated Annealing

Simulated annealing generalizes stochastic local search. With some probability, say p, simulated annealing accepts state changes that adversely affect the value of the objective function. In other words, in accordance with p, it encourages *exploration* of harmful states; otherwise, it *exploits* successful states.

A probability p of exploration can be determined in several ways. One idea is simply to fix p at some small value, say $\epsilon > 0$. Another approach is to let p decrease with time: $p \sim 1/t$. Yet another option is to let p decrease as $\Delta(x, y) = f(y) - f(x)$ increases, where x is the current solution and $y \in \mathcal{N}(x)$.

Updating in the spirit of this third idea can be achieved as follows: if $\Delta(x, y) < 0$ (i.e., y is an improvement), then let $p = 1$: i.e., exploit; if $\Delta(x, y) \geq 0$ (i.e., y is not an improvement), then let $p = e^{-\Delta(x,y)}$. In this way, small increases in the value of the objective function imply large values of p (exploration is likely; exploitation is not), whereas large increases in the value of the objective function imply small values of p (exploitation is likely; exploration is not). (For this definition of p, all ties are broken in favor of y, since $e^0 = 1$.)

Simulated annealing is a local search algorithm based on a combination of ideas: (i) exploration decreases with time and (ii) exploration is more likely if it makes things only slightly worse, and less likely if it makes things significantly worse. Specifically, simulated annealing relies on the following exploration probabilities, given by the **Boltzmann distribution**: for **temperature** $\tau > 0$,

$$p = \begin{cases} e^{-\Delta(x,y)/\tau} & \text{if } \Delta(x, y) \geq 0 \\ 1 & \text{otherwise} \end{cases}$$

Simulated annealing derives its name from the interpretation of τ as a temperature, which through slow cooling strengthens metal during its production. Initially, when $\tau \gg 0$ and p is high, simulated annealing behaves like a random walk: exploration is more likely than exploitation. Later in the search, when $\tau \to 0$ and p is low, simulated annealing reduces to stochastic local search: exploitation is more likely than exploration. (See Figure 4.)

A typical decay schedule for the temperature parameter is as follows: for some cooling constant c,

$$\tau_t = \frac{c}{\ln(t + 1)}$$

FIRST($X, f, \mathcal{N}, p, x, \epsilon$)	
Inputs	optimization problem
	patience time limit p
	random start state x
	rate of exploration ϵ
Output	best state visited x
Initialize	$t = 0$

`while` $(t < p)$ `do`

1. for some $y \in \mathcal{N}(x)$
   (a) if $f(y) < f(x)$, $x = y$, then $t = 0$
   (b) else if $f(y) = f(x)$ and `rand`$[0, 1] \leq \epsilon$, then $x = y$ and increment t
   (c) else increment t

`return` x

Table 2: First-Improvement Search.

RANDOM($X, f, \mathcal{N}, p, n, \epsilon$)	
Inputs	optimization problem
	patience time limit p
	number of restarts n
	rate of exploration ϵ
Output	best state visited x^*
Initialize	$f(x^*) = \infty$

`for` $i = 1$ `to` n

1. choose start state $x \in X$
2. $z =$ FIRST$(X, f, \mathcal{N}, p, x, \epsilon)$
3. if $f(z) < f(x^*)$, then $x^* = z$

`return` x^*

Table 3: Stochastic Local Search with Random-Restarts.

Simulated annealing is **asymptotically optimal** (i.e., in the limit of infinite iterations) as long as c is set conservatively, i.e., large enough, so that the temperature never cools too fast. In theory, this condition guarantees that simulated annealing can escape all local optima with some positive probability, and thus the probability of eventually discovering a global optimum approaches 1. In practice, however, this decay schedule is far too slow to be useful. A practical alternative is a geometric cooling schedule of the form: $\tau_t \leftarrow \alpha\tau_{t-1}$, for some $\alpha \in (0, 1)$. Typical choices for α are 0.95 and 0.99.

Remark The exploration vs. exploitation tradeoff arises whenever exploration is costly in the short term but could be beneficial in the long term. This phenomenon arises in any area of AI that embodies decision making under uncertainty *and* where actions are informative. For example, a recommender system that plans to show Professor Greenwald a bicycle ad has to decide whether to show her an ad for a Trek bicycle, an act of exploitation assuming it has seen her purchase a Trek in the past, or an ad for a Cannondale, an act of exploration, since she has never purchased a Cannondale.

SA$(X, f, \mathcal{N}, T, x, C, \tau)$	
Inputs	optimization problem
	number of iterations T
	random start state x
	cooling schedule C
	temperature τ
Output	best state visited x^*
Initialize	$x^* = x$, τ according to C

`for` $t = 1$ `to` T

1. for some $y \in \mathcal{N}(x)$
   (a) compute $\Delta(x, y) = f(y) - f(x)$
   (b) if $\Delta(x, y) < 0$, let $p = 1$
   (c) else $p = e^{-\Delta(x,y)/\tau}$
   (d) if `rand`$[0, 1] \leq p$, $x = y$
      i. if $f(x) < f(x^*)$, $x^* = x$
2. decay τ according to C

`return` x^*

Table 4: Simulated Annealing.

We will encounter the exploration-exploitation tradeoff again when we study reinforcement learning.

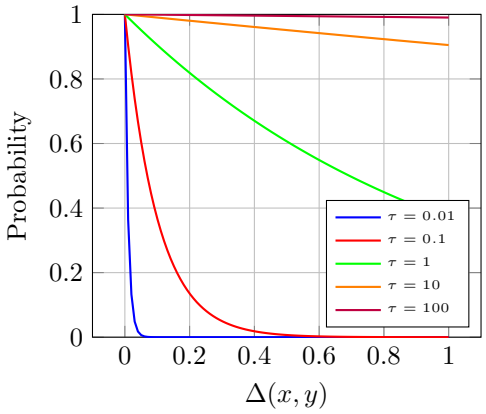


Figure 4: Boltzmann Distribution ($e^{-\Delta(x,y)/\tau}$) at various temperatures τ

A TSP as Search

Given a set of cities Y of cardinality m, and given the corresponding distances between cities ($d(x, y)$, for all $x, y \in Y$), we formulate TSP as a search problem in two ways. In the first formulation, states are **Hamiltonian paths**: paths that do not visit any city twice. In the second formulation, states are

Hamiltonian circuits: cycles that do not visit any city twice.

A.1 Hamiltonian Paths

Let the set of states X consists of all Hamiltonian *paths*: i.e., paths that do not visit any city twice. Formally,

$$X = \{(x_1, \ldots, x_n) \mid n = 1, \ldots, m+1, x_i \in Y \text{ for all } 1 \leq i \leq n, \\ x_i \neq x_j \text{ for all } 1 \leq i \neq j \leq n, \text{ unless } i = 1 \text{ or } j = m+1\}$$

The set of start states includes all states of length 1, while the set of goal states includes all states of length $m+1$. The transition function between states is defined as follows: at state $(x_1, \ldots, x_n)$,

$$\mathcal{T}(x_1, \ldots, x_n) = \{(x_1, \ldots, x_n, x_{n+1}) \mid x_{n+1} \neq x_i \in Y, \text{ for all } 1 \leq i \leq n, \} \\ \text{unless } i = 1, n = m$$

with cost function $c((x_1, \ldots, x_n), (x_1, \ldots, x_n, x_{n+1})) = d(x_n, x_{n+1})$. Figure 5 depicts the first three levels of the search space in this formulation for the traveling salesperson problem depicted in Figure 2.

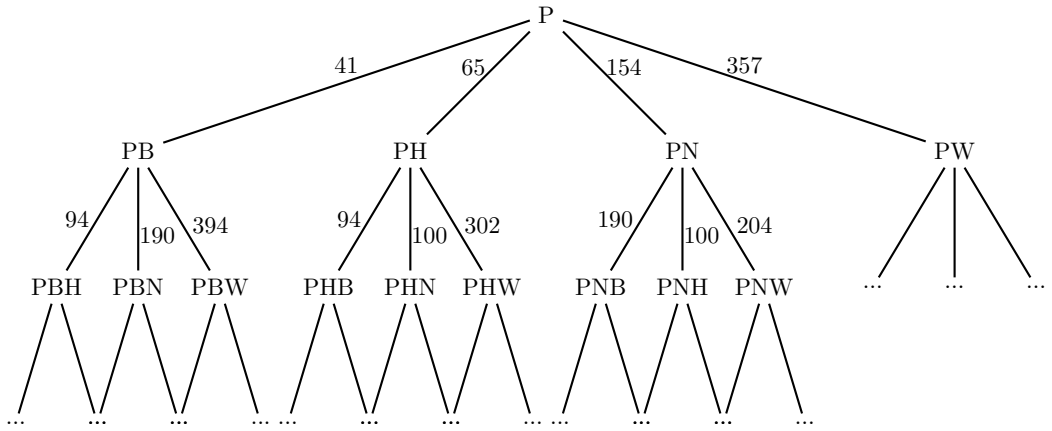


Figure 5: Search in TSP: States are Hamiltonian Paths.

One simple heuristic to solve TSP in this formulation is **nearest-neighbor**. From city x, the salesperson visits a city $y \in \mathcal{T}(x)$ *s.t.* $d(x, y)$ is minimal and y has not yet been visited. Applying this heuristic to the search problem in Figure 5 generates the suboptimal path PBHNWP of distance 796.

A.2 Hamiltonian Circuits

Let the set of states X consists of all Hamiltonian *circuits*: i.e., cycles that do not visit any city twice—*a.k.a* tours. Formally,

$$X = \{(x, y_1, \ldots, y_n, x) \mid n = 0, \ldots, m-1, \\ x \neq y_1, y_i \neq y_j, y_n \neq x, \text{ for all } y_j \in Y \text{ and } 1 \leq i \neq j \leq n\}$$

In this case, the transition function between states is defined as follows: at state $(x, y_1, \ldots, y_i, y_{i+1}, \ldots, y_n, x)$,

$$\mathcal{T}(x, y_1, \ldots, y_n, x) = \{(x, y_1, \ldots, y_i, y, y_{i+1} \ldots, y_n, x) \mid y \neq y_i \text{ and } y_i \neq x, \text{ for all } y, y_i \in Y\}$$

at cost $d(y_i, y) + d(y, y_{i+1}) - d(y_i, y_{i+1})$. The set of goal states includes all states of length $m+1$. The set of start states includes all states of length 2.

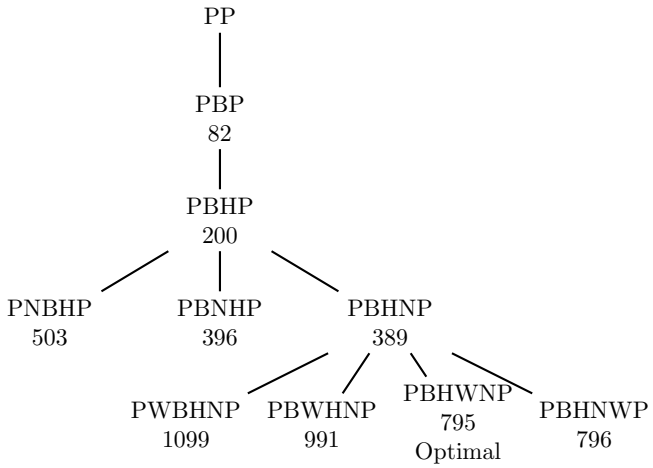


Figure 6: Search in TSP: States are Hamiltonian circuits.

A greedy heuristic for solving TSP in this problem formulation is simply: insert a city into the tour that causes the smallest increase in the length of the tour. This heuristic finds an optimal tour in our example. (See Figure 6.)